



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251176
Nama Lengkap	David Alexsi Cahyadi
Minggu ke / Materi	13 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Tuple Immutable

Tuple merupakan tipe data dalam Python yang memiliki kemiripan dengan list. Nilai yang disimpan di dalamnya berupa tipe data apapun dan diakses menggunakan indeks bilangan bulat (integer). Namun, perbedaan mendasar antara tuple dan list terletak pada sifat **immutable** yang dimiliki tuple, artinya elemen-elemen di dalamnya tidak dapat diubah setelah tuple dibuat.

Selain itu, tuple bersifat **hashable**, yaitu memiliki nilai hash yang tidak pernah berubah selama objek tersebut ada. Sifat ini memungkinkan tuple untuk digunakan sebagai *key* dalam dictionary, sesuatu yang tidak bisa dilakukan oleh list.

Penulisan tuple dapat dilakukan dengan beberapa cara. Yang paling sederhana adalah dengan memisahkan nilai menggunakan tanda koma:

```
>>> t = 'a','b','c','d','e'
```

Bisa juga ditulis menggunakan tanda kurung:

```
>>> t = ('a','b','c','d','e')
```

Untuk tuple yang berisi satu elemen, tanda koma tetap harus ditambahkan di belakang agar Python mengenalinya sebagai tuple, bukan string biasa:

```
>>> t1 = ('a',)    >>> t2 = ('a')
>>> type(t1)       >>> type(t2)
<type 'tuple'>    <type 'str'>
```

Selain itu, tersedia juga fungsi bawaan **tuple()** untuk membuat tuple kosong atau mengkonversi urutan (seperti string atau list) menjadi tuple:

```
>>> t = tuple('dutawacana')
>>> print(t)
('d','u','t','a','w','a','c','a','n','a')
```

Meskipun elemen tuple tidak bisa diubah secara langsung, kita tetap bisa “mengganti” isi tuple dengan cara membuat tuple baru yang menggabungkan elemen yang diinginkan:

```
>>> t = ('A',) + t[1:]
>>> print(t)
('A', 'b', 'c', 'd', 'e')
```

Membandingkan Tuple

Operator perbandingan pada Python dapat bekerja pada tuple maupun tipe sekuensial lainnya seperti list dan dictionary. Cara kerjanya adalah dengan membandingkan elemen pertama dari masing-masing tuple terlebih dahulu. Jika elemen pertama sama, maka perbandingan dilanjutkan ke elemen berikutnya, dan seterusnya hingga ditemukan perbedaan.

```
>>> (0, 1, 2) < (0, 3, 4)
True
>>> (0, 1, 2000000) < (0, 3, 4)
True
```

Konsep perbandingan ini juga menjadi dasar cara kerja fungsi **sort()** di Python, yang dikenal dengan pola **DSU (Decorate, Sort, Undecorate)**:

- **Decorate**, membangun list of tuple dengan key pengurutan sebelum elemen utama.
- **Sort**, mengurutkan list of tuple menggunakan **sort()**.
- **Undecorate**, mengekstrak elemen yang sudah diurutkan kembali ke dalam list biasa

Sebagai contoh, berikut adalah program yang mengurutkan kata-kata dalam sebuah kalimat dari yang paling panjang ke yang paling pendek:

```
1
2 kalimat = 'but soft what light in yonder window breaks'
3 dafkata = kalimat.split()
4 t = list()
5 for kata in dafkata:
6     t.append((len(kata), kata))
7
8 t.sort(reverse=True)
9
10 urutan = list()
11 for length, kata in t:
12     urutan.append(kata)
13
14 print(urutan)
15
16
```

Outputnya dari program tersebut adalah:

```
['yonder', 'window', 'breaks', 'light', 'what',  
'soft', 'but', 'in']
```

Kata-kata tampil diurutkan berdasarkan panjangnya, dari yang terpanjang ke yang terpendek. Apabila ada dua kata dengan panjang yang sama, keduanya diurutkan secara alfabet.

Penugasan Tuple

Salah satu fitur menarik dari Python adalah kemampuan menempatkan tuple di sisi kiri dari sebuah pernyataan penugasan (*assignment*). Fitur ini memungkinkan kita untuk menetapkan nilai ke lebih dari satu variabel sekaligus dalam satu baris.

Sebagai contoh, jika kita memiliki list dua elemen, kita bisa langsung membagi isinya ke dalam dua variabel:

```
>>> m = [ 'have', 'fun' ]  
>>> x, y = m  
>>> x  
'have'  
>>> y  
'fun'  
>>>
```

Python secara internal menerjemahkan sintaks tersebut menjadi:

```
>>> m = [ 'have', 'fun' ]  
>>> x = m[0]  
>>> y = m[1]
```

Penulisan dengan tanda kurung juga diperbolehkan dan menghasilkan hasil yang sama:

```
>>> m = [ 'have', 'fun' ]  
>>> (x, y) = m
```

Fitur penugasan tuple ini juga sangat berguna untuk menukar nilai dua variabel hanya dalam satu baris tanpa memerlukan variabel sementara:

```
>>> a, b = b, a
```

Yang perlu diperhatikan, jumlah variabel di sisi kiri harus selalu sama dengan jumlah elemen di sisi kanan. Jika tidak, Python akan menampilkan error **ValueError: too many values to unpack**.

Penugasan tuple juga sering dipakai untuk memisahkan bagian-bagian dari sebuah string, misalnya memecah alamat email menjadi username dan domain:

```
>>> email = 'didanendya@ti.ukdw.ac.id'
>>> username, domain = email.split('@')
```

Dictionaries and Tuple

Dictionary di Python memiliki method bernama **.items()** yang mengembalikan list of tuple, di mana setiap tuple berisi pasangan *key-value* dari dictionary tersebut.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('b', 1), ('a', 10), ('c', 22)]
```

Karena hasil **.items()** berbentuk list of tuple, kita bisa menerapkan fungsi **sort()** untuk mengurutkan isinya. Secara default, pengurutan dilakukan berdasarkan elemen pertama tuple, yaitu *key* dari dictionary. Ini adalah cara yang efektif untuk menampilkan isi dictionary dalam urutan yang teratur:

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

List yang dihasilkan diurutkan secara *ascending* berdasarkan alfabet dari *key* nya.

Multipenugasan dengan dictionaries

Kombinasi antara method `.items()`, penugasan tuple, dan perulangan `for` menghasilkan cara yang ringkas dan elegan untuk mengakses key sekaligus value dari dictionary dalam satu iterasi:

```
for key, val in list(d.items()):
    print(val, key)
```

Pada setiap iterasi, Python secara otomatis melakukan *tuple assignment*, memecah setiap pasangan (**key**, **value**) dari `.items()` dan memasukkannya ke variabel **key** dan **val** bersamaan.

Teknik ini bisa dikembangkan lebih lanjut untuk mencetak isi dictionary yang diurutkan berdasarkan nilainya (bukan key nya). Caranya adalah dengan membuat list of tuple dalam format (**value**, **key**) terlebih dahulu, kemudian mengurutkannya secara terbalik:

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) ) ..
>>> l
[(10, 'a'), (22, 'c'), (1, 'b')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
>>>
```

Dengan menempatkan value sebagai elemen pertama tuple, fungsi `sort()` akan otomatis mengurutkan berdasarkan value tersebut.

Kata yang sering muncul

Pada bagian ini, konsep-konsep yang telah dipelajari sebelumnya diterapkan dalam senilai studi kasus nyata misalnya, menghitung dan menampilkan **10 kata yang paling sering muncul** dalam sebuah file teks, yaitu naskah *Romeo and Juliet Act 2, Scene 2*.

Program bekerja dengan membaca file baris per baris, membersihkan tanda baca, mengubah semua huruf menjadi huruf kecil, lalu menghitung frekuensi tiap kata ke dalam dictionary. Selanjutnya, dictionary dikonversi menjadi list of tuple berformat **(frekuensi, kata)**, diurutkan secara menurun, dan dicetak 10 teratas menggunakan slicing **lst[:10]**.

```
import string
fhand = open('romeo-full.txt')
counts = dict()
for line in fhand:
    line = line.translate(str.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

# urutkan dictionary by value
lst = list()
for key, val in list(counts.items()):
    lst.append((val, key))

lst.sort(reverse=True)

for key, val in lst[:10]:
    print(key, val)
```

```
61 1
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
```

Output yang dihasilkan menampilkan kata-kata paling dominan seperti “i”, “and”, dan “romeo” beserta jumlah kemunculannya. Hal ini membuktikan betapa ringkasnya Python dalam menyelesaikan analisis teks yang cukup kompleks hanya dalam sekitar 19 baris kode.

Tuple sebagai kunci dictionaries

Karena tuple bersifat *hashable*, tuple dapat digunakan sebagai key dalam dictionary, termasuk sebagai *composite key*, kunci yang terdiri dari lebih dari satu informasi.

Contohnya adalah membuat direktori telepon yang memetakan kombinasi nama belakang dan nama depan ke nomor telepon. Tuple **(last, first)** digunakan langsung sebagai key, dan data dapat diakses kembali menggunakan perulangan **for** dengan penugasan tuple secara bersamaan.

```
>>> last = 'nendya'
>>> first = 'dida'
>>> number = '088112266'
>>> directory = dict()
>>> directory[last, first] = number
>>> for last, first in directory:
...     print(first, last, directory[last,first])
...
dida nendya 088112266
```

```
for last, first in directory:
    print(first, last, directory[last,first])
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github: https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 11.1

Soal:

Latihan 11.1 Buatlah program untuk melakukan pengecekan apakah semua anggota yang ada didalam tuple sama.

Contoh:

```
tA= (90, 90, 90, 90)
```

Output

True

Source Code:

```
tA = eval(input("Masukkan tuple: "))
hasil = len(set(tA)) == 1
print (hasil)
```

Output:

```
PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal01.py"
Masukkan tuple: (90,90, 90, 90)
True
PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal01.py"
Masukkan tuple: (90,90, 90, 80)
False
PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal01.py"
Masukkan tuple: (123,123, 123, 123)
True
```

Penjelasan:

Program dimulai dengan menerima input dari pengguna menggunakan fungsi **input()** yang hasilnya berupa string, kemudian fungsi **eval()** langsung mengkonversi string tersebut menjadi tipe data tuple sesuai format yang dimasukkan pengguna, misalnya **(90, 90, 90, 90)**.

Selanjutnya, pengecekan kesamaan elemen. Tuple yang telah dibuat dikonversi terlebih dahulu menjadi **set** menggunakan fungsi **set()**. Karena set secara otomatis menghapus elemen duplikat, maka jika semua anggota tuple bernilai sama, set hanya akan menyisakan satu elemen unik. Berikutnya, fungsi **len()** menghitung jumlah elemen dalam set tersebut, lalu dibandingkan dengan angka 1. Jika hasilnya sama dengan 1 maka program mencetak **True**, yang berarti semua anggota tuple identik. Sebaliknya jika terdapat lebih dari satu elemen unik dalam set, program akan mencetak **False**.

Latihan 11.2

Soal:

Latihan 11.2 Buatlah program dengan menggunakan tuple yang dapat melakukan proses seperti pada kasus 11.1, Gunakan data diri anda masing-masing dan lakukan perubahan supaya didapatkan output seperti contoh berikut ini :

Contoh:

Data: ('Matahari Bhakti Nendya', '22064091', 'Bantul, DI Yogyakarta')

NIM : 22064091

NAMA : Matahari Bhakti Nendya

ALAMAT : Bantul, DI Yogyakarta

NIM: ('2', '2', '0', '6', '4', '0', '9', '1')

NAMA DEPAN: ('a', 't', 'a', 'h', 'a', 'r', 'i')

NAMA TERBALIK: ('Nendya', 'Bhakti', 'Matahari')

Source Code:

```
data = eval(input("Masukkan Data: "))
print("Data:", data)
print()

na,ni,al = data
print("NIM :",ni)
print("NAMA :",na)
print("ALAMAT :",al)
print()
```

```

tni= tuple(ni)
print("NIM:", tni)
print()

nama = tuple(na.split()[0][1:])
print("NAMA DEPAN:", nama)
print()

aman = tuple(na.split()[::-1])
print("NAMA TERBALIK:", aman)

```

Output:

```

PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal02.py"
Masukkan Data: ('MatahariBhakti Nendya', '22064091', 'Bantul,DI Yogyakarta')
Data: ('MatahariBhakti Nendya', '22064091', 'Bantul,DI Yogyakarta')

NIM : 22064091
NAMA : MatahariBhakti Nendya
ALAMAT : Bantul,DI Yogyakarta

NIM: ('2', '2', '0', '6', '4', '0', '9', '1')

NAMA DEPAN: ('a', 't', 'a', 'h', 'a', 'r', 'i', 'B', 'h', 'a', 'k', 't', 'i')

NAMA TERBALIK: ('Nendya', 'MatahariBhakti')
PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal02.py"
Masukkan Data: ('Wisnu ciong', '08523321', 'Hubei,Wuhan')
Data: ('Wisnu ciong', '08523321', 'Hubei,Wuhan')

NIM : 08523321
NAMA : Wisnu ciong
ALAMAT : Hubei,Wuhan

NIM: ('0', '8', '5', '2', '3', '3', '2', '1')

NAMA DEPAN: ('i', 's', 'n', 'u')

NAMA TERBALIK: ('ciong', 'wisnu')

```

Penjelasan:

Program dimulai dengan menerima input dari pengguna menggunakan fungsi **input()**, kemudian fungsi **eval()** mengonversi string yang dimasukkan menjadi tipe data tuple secara otomatis. Tuple tersebut disimpan dalam variabel **data** dan langsung dicetak sebagai baris pertama output.

Selanjutnya, program melakukan penugasan tuple dengan memecah isi **data** ke dalam tiga variabel sekaligus yaitu **na** untuk nama, **ni** untuk NIM, dan **al** untuk alamat. Ketiga variabel tersebut kemudian dicetak satu per satu sesuai labelnya.

Untuk mencetak NIM dalam bentuk tuple karakter, variabel **ni** dikonversi menggunakan fungsi **tuple()** sehingga setiap karakter NIM menjadi elemen tersendiri dalam tuple baru **tni**.

Untuk mendapatkan nama depan tanpa huruf pertama, program menggunakan **na.split()[0]** untuk mengambil kata pertama dari nama, kemudian **[1:]** untuk memotong karakter pertamanya. Hasilnya dikonversi menjadi tuple menggunakan **tuple()** dan disimpan dalam variabel **nama**.

Untuk mencetak nama secara terbalik, program menggunakan **na.split()** untuk memisahkan nama menjadi list kata-kata, kemudian **[::-1]** untuk membalik urutan kata tersebut. Hasilnya dikonversi menjadi tuple dan disimpan dalam variabel **aman**, lalu dicetak sebagai output terakhir.

Latihan 11.3

Soal:

Latihan 11.3 Buatlah program untuk menghitung distribusi jam dalam satu hari dimana ada pesan yang diterima dari setiap email yang masuk. Gunakan file mbox-short.txt untuk sebagai datanya. Berikut ini adalah contoh output dari programnya.

Contoh:

Enter a file name: mbox-short.txt

04 3

06 1

07 1

09 2

10 3

11 6

14 1

15 2

16 4

17 2

18 1

19 1

Source Code:

```
kamus = dict()
file = input("Enter a file name: ")
try:
    buka = open(file)
except FileNotFoundError:
    print("File not found:", file)
    exit()

for i in buka:
    kata = i.split()
    if len(kata) < 2 or kata[0] != 'From':
        continue
    else:
        jam = kata[5].split(':')[0]
        if jam not in kamus:
            kamus[jam] = 1
        else:
            kamus[jam] += 1
ubah = list()
for k, v in list(kamus.items()):
    ubah.append((k, v))
ubah.sort()
```

```
for jam, jumlah in ubah:
    print(jam, jumlah)
```

Output:

```
PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal03.py"
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
PS D:\PrAlpro\Pertemuan 13> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 13/soal03.py"
Enter a file name: kamus-bahasa-masphilip.txt
File not found: kamus-bahasa-masphilip.txt
```

Penjelasan:

Program dimulai dengan membuat dictionary kosong **kamus** untuk menyimpan data distribusi jam. Kemudian program meminta input nama file dari pengguna. Fungsi **open()** digunakan untuk membuka file tersebut dalam blok **try**, dan jika file tidak ditemukan maka blok **except FileNotFoundError** akan mencetak pesan error dan menghentikan program menggunakan **exit()**.

Selanjutnya program melakukan perulangan pada setiap baris file menggunakan variabel **i**. Setiap baris dipecah menjadi list kata-kata menggunakan **split()** dan disimpan dalam variabel **kata**. Jika jumlah kata kurang dari 2 atau kata pertama bukan **'From'**, baris tersebut dilewati menggunakan **continue**. Hanya baris yang diawali **'From'** yang diproses lebih lanjut, karena baris tersebut mengandung informasi pengirim dan waktu pengiriman email.

Dari baris yang lolos filter, jam diambil dari elemen ke 5 (**kata[5]**) yang berformat **09:14:16**, kemudian dipotong menggunakan **split(':')[0]** untuk mengambil bagian jamnya saja. Jam tersebut kemudian dihitung frekuensinya ke dalam dictionary **kamus**. Jika jam belum ada

dalam dictionary maka diinisialisasi dengan nilai 1, jika sudah ada maka nilainya ditambah 1.

Setelah semua baris selesai diproses, dictionary **kamus** dikonversi menjadi list of tuple menggunakan **.items()** dengan format **(jam, jumlah)** dan disimpan dalam list **ubah**. List tersebut kemudian diurutkan secara ascending menggunakan **sort()** berdasarkan jam. Terakhir, program mencetak setiap pasangan jam dan jumlah pesannya menggunakan penugasan tuple **jam, jumlah** dalam perulangan **for**.